CS 300 Module Five – Binary Search Tree: Code Reflection and Pseudocode

Code Reflection

The purpose of this program is to store auction bids from a municipal eBid CSV file in a binary search tree keyed on the bid ID, so that bids can be loaded, displayed in sorted order, searched, and removed. The tree is built from a `Node` structure (a `Bid` plus left and right pointers) and a `BinarySearchTree` class that keeps a `root` pointer and a `size` counter as housekeeping variables. Insert and remove use recursion (`addNode`, `removeNode`), search uses a simple loop that walks down the tree, and display uses a recursive in-order traversal, which prints the bids in sorted order by ID. Because each comparison discards half of the remaining tree, searching the 12,023-bid file takes only a handful of clock ticks, compared with scanning a list.

The main challenge was `removeNode`, which has to handle four cases: a leaf, a node with only a left child, a node with only a right child, and a node with two children. For the two-child case I replace the node's bid with its in-order successor (the smallest bid in the right subtree) and then recursively delete that successor, which keeps the search-tree ordering intact. A second challenge was making sure pointers are re-linked correctly: each recursive call returns the new subtree root, and the caller assigns it back (`node->left = removeNode(...)`), so deleted nodes never leave dangling pointers. I also had to forward-declare `displayBid` so the traversal methods could call it, and I added a destructor that deletes all nodes post-order to avoid memory leaks. I tested with both CSV files and with a small tree covering each removal case.

Pseudocode

```
class BinarySearchTree
    root <- nullptr
    size <- 0

Insert(bid)
    if root is nullptr
        root <- new Node(bid)
    else
        addNode(root, bid)
    size <- size + 1

addNode(node, bid)
    if node.bidId > bid.bidId
        if node.left is nullptr
            node.left <- new Node(bid)
        else
            addNode(node.left, bid)
```

```
    else
        if node.right is nullptr
            node.right <- new Node(bid)
        else
            addNode(node.right, bid)

Search(bidId)
    current <- root
    while current is not nullptr
        if current.bidId == bidId
            return current.bid
        if bidId < current.bidId
            current <- current.left
        else
            current <- current.right
    return empty Bid                    // empty bidId means "not found"

Remove(bidId)
    root <- removeNode(root, bidId)

removeNode(node, bidId)
    if node is nullptr
        return nullptr                  // bid not in tree
    if bidId < node.bidId
        node.left <- removeNode(node.left, bidId)
    else if bidId > node.bidId
        node.right <- removeNode(node.right, bidId)
    else                                // match found
        if node has no children
            delete node; size <- size - 1
            return nullptr
        else if node has only a left child
            temp <- node.left; delete node; size <- size - 1
            return temp
        else if node has only a right child
            temp <- node.right; delete node; size <- size - 1
            return temp
        else                            // two children
            temp <- node.right
            while temp.left is not nullptr
                temp <- temp.left       // in-order successor
            node.bid <- temp.bid
            node.right <- removeNode(node.right, temp.bidId)
    return node

inOrder(node)
    if node is not nullptr
        inOrder(node.left)
        display node.bid
        inOrder(node.right)

(postOrder and preOrder are identical except the display step moves
 after / before the two recursive calls.)

~BinarySearchTree()
    destroyTree(root)
destroyTree(node)
```

```
if node is not nullptr
    destroyTree(node.left)
    destroyTree(node.right)
    delete node
```